

Artificial Intelligence Final Report

Tucker Coughlin

May 2023

1 Introduction

This project looked at various artificial intelligence techniques for creating a Checkers player. The scope of this project was for a 3-month undergraduate course at the 300 level.

The project was split into 3 phases:

1. Creating a checkers game with a simple GUI,
2. Creating a Search Tree algorithm with alpha-beta pruning.
3. Creating and training a temporal difference SARSA reinforcement learner with a linear function approximator.

Using Checkers as the model of a perfect information game was influenced by the abilities of the artificial intelligence players. I wanted to have players that were good enough to make reasonable moves in the game, but not so good that they only make ideal moves. In this middle ground, players can have noticeable improvements based on the parameters of the algorithms. Changing aspects of the code leads to quantifiable results that would not be possible if the game was too difficult or too easy.

2 Search Tree with Alpha-Beta Pruning

I implemented a standard version of the min-max algorithm with alpha-beta pruning (abbreviated AB-player from now on). The AB-player can search up to depth 11 in a reasonable amount of time (usually 5 to 30 second range). The time it takes to evaluate the search tree increases slightly as the game progresses as there are more moves available to the AB-player.

2.1 Evaluation:

The evaluation function simply adds the number of its own pieces and subtracts the number of the opponent's pieces (with pawns being worth 1 point and kings being worth 2). A winning end game is worth 100 points and a losing end game is worth -100 points. I didn't use infinity to keep the scores an integer data type, but 100 is higher than the maximum score that could be achieved by normal play (24) so it has the same effect.

2.2 In game performance:

This simple evaluation seems to work extremely well for checkers. The outcome is a very aggressive player who is obsessed with capturing pieces and making kings. I observed the AB-player carefully setup double jumps and create traps for its opponents. The algorithm dominates especially in the early game when it is easy to capture pieces and gains a quick advantage. However, it struggles slightly in the late game, especially when there are no ways to capture pieces within its search tree. At this point it will just make random moves, wandering around the board until the opponent is within reach.

2.3 Tests:

This AB-player is good at checkers. In a simple test of 5 games against undergraduate college students, only one student was able to beat the AB-player at search depth 11. In the game that a human won, the AB-player was doing very well in the first half of the game but started to make bad decisions in the late game when there was no clear way to take pieces or achieve victory. This gave the human player time to arrange their pieces into a strong structure and plan out an attack that the AB-player couldn't respond to. I myself have not been able to beat the AB-player at any search depth above 6, but I am not known for my checkers skills.

2.4 Improvements for the future:

I could implement a variable search depth that searches deeper in the tree if all options have the same outcome value (meaning there is no way to take new pieces), or decreases the search depth if it finds obviously good moves early in the search (even if the moves aren't perfectly ideal). I could also add more aspects to the evaluation function

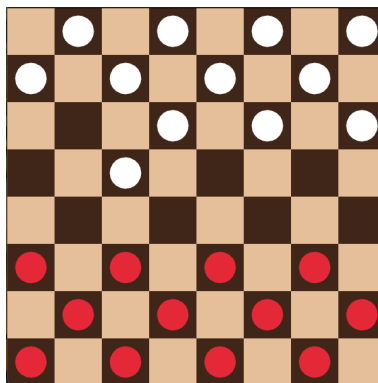


Figure 1: The game's GUI. The AB-Player makes its first move

such as a need to: move pieces into the opponent's territory, keep a strong backline, or make strong formations in the middle of the board.

3 Temporal Difference Reinforcement Learning

I implemented the SARSA (State, Action, Reward, State, Action) algorithm for this player (abbreviated RL-player from now on). The RL-player uses a linear function approximator, which evaluates the board for these parameters:

1. The number of white pawns
2. The number of red pawns
3. The number of white kings
4. The number of red kings
5. The number of white pieces on the backline (top row of the board)
6. The number of red pieces on the backline (bottom row of the board)
7. The number of adjacent white pieces (pieces positioned diagonally together)
8. The number of adjacent red pieces (pieces positioned diagonally together)
9. The number of white pieces in the center of the board (the middle 4x4 squares)
10. The number of red pieces in the center of the board (the middle 4x4 squares)

3.1 Implementation decisions:

The training process required a lot of tweaking and careful adjustment of the algorithm parameters. I in this next section I will cover the values I used for each SARSA parameter, issues I ran into in training and how I fixed them, and the methodology behind my choices.

3.1.1 Epsilon values:

The algorithm uses an epsilon-greedy approach during training, taking greedy actions move $1-\epsilon$ % of its turns, and making a random "exploratory" move ϵ % of its turns.

Note: My RL program has ϵ backwards from the standard literature. In my code, when ϵ is 0.9 the algorithm will take greedy actions 90% of its turns and explore 10% of its turns. Whereas in standard literature an ϵ of 0.9 means the algorithm will take greedy actions 10% of its turns and explore 90% of its turns. I made this decision to simplify the implementation of the algorithm. In this report, I use the standard definition of epsilon even though it is contrary to my code.

At the start of training, the RL-player is very bad at checkers because its evaluation function values arbitrary positions. It would often go back and forth making the same move repeatedly, only making a novel choice when it was exploring epsilon percent of its turns. The outcome of this is very long games, where both players get stuck until they make a random move. To decrease training time I started epsilon to be very high, meaning it will explore more often at first. The idea of varying the value of epsilon temporally by $1/t$ is briefly suggested in Sutton and

Barto page 129. I took this general idea, but instead decrease ϵ over a series of episodes instead of a singular episode. I found 0.5 to be a good starting value. I then subtract a constant each episode until $\epsilon = 0$.

3.1.2 Discounting:

I didn't do it. I don't care how long the RL-player takes to win the game as long as it eventually does. The games are episodic so this did not present an issue.

3.1.3 Non-convergence:

Another issue I found is that the weights for each feature in the linear function approximator would approach infinity as training progresses. There are two possible reasons for this behavior.

My first hypothesis for this behavior is that the true value of each state-action pair is impossible (or at least very difficult) to represent with floating-point numbers. The values of the weights are proportional to all the other weights in the vector. For each state looked at: $\text{argmax}_a \langle \mathbf{X}(s, a), \mathbf{W} \rangle = \text{argmax}_a \langle \mathbf{X}(s, a), \mathbf{W}, \mathbf{C} \rangle$ where $\mathbf{C}$ is a vector filled with some constant. This means the entire weight vector can grow and shrink, while still representing the true values of each state/action pair.

This is important because the weights can continue to grow towards infinity while still converging to the true policy.

As an example of how this can be an issue: if the true values of weights W_1 and W_2 are $W_1 = W_2$ then:

$W_2 > W_1 \implies \text{increase}(W_1)$ and

$W_2 < W_1 \implies \text{increase}(W_2)$

The system will only reach equilibrium when the values of the weights are perfectly balanced, which will never happen because: 1. limitations of floating-point values and 2. The constant step size reacting to new information. The result is a feedback loop where:

$\text{increase}(W_1) \implies W_1 > W_2 \implies \text{increase}(W_2) \implies W_1 < W_2$

And the cycle repeats.

The second hypothesis is related to the step size. For my training I am using a constant step size. A constant step size does not ensure convergence to the true values of the weights in the future vector. "for the case of constant step-size parameter ... the estimates never completely converge but continue to vary in response to the most recently received rewards" (Sutton and Barto, page 33) Because this is a stationary environment, this is not ideal behavior. I could have fixed this issue by decreasing the step size as training continued but decided not to because the game is, what I call, "semi-non-stationary" in the fact that the untrained RL-player is training against another untrained RL-player. The strategies learned to beat the untrained RL-player should not be the same strategies used once training is complete, meaning I want the weights to respond to new strategies it encounters during training. So I decided to have a constant step size, even though a sample average method would have been more ideal for the stationary task of a perfect information game.

To fix the convergence issue I put a maximum value on the weights, which only works if the maximum value is high enough that the weights will approach their true values before any one hits that maximum. I think that this might be an issue with the learner because some weights can't grow properly and therefore aren't valued as highly as they should be. I just made sure to keep this maximum high to give the weights "room" to properly compare to each.

A more mature approach would be to periodically normalize all the weights to keep them small while also maintaining their proportions to each other.

3.2 Training:

I trained this RL-Player for 1 million games. The Red player and the White player were trained as separate agents and used the exact same feature vector.

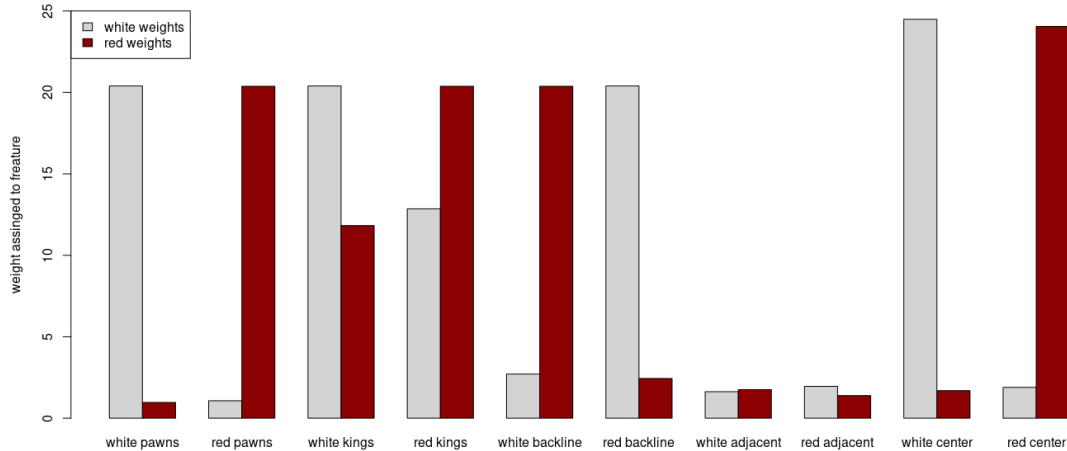


Figure 2: The weights assigned to each feature after 1 million games of training

The RL-player quickly figured out that it values moves where it has more of its own pieces and less of the opponents' pieces. It values having pieces in the center very highly, above anything else, which ends up being what cripples it most during actual play. It doesn't value having kings as highly as it values pawns, possibly because it isn't as common throughout the entire game and the feature values are lower in general. Both the red RL-player and the white RL-player approached the same weight vectors, just flipped for their own pieces.

3.3 Tests:

Any human player could beat the RL-player, and the AB-player absolutely crushes the RL-player in 100% of the games they played (search depth ≥ 4). The RL-player values being in the center too much. It also can't come up with different strategies for early vs late game, which could be a future direction for the project.

3.4 Future Directions:

I have already started a new version of the RL-Player that's fixing some of the most glaring issues with the current version. The main things I'd like to change are:

- 1 Normalizing the values in the feature vector
- 2 Taking into account even more features of the actions to associate with the state
- 3 Using normalization instead of a maximum value on the weight vector to keep the values low without disrupting the proportions of the weight vector
- 4 Training the RL-Player against the AB-Player. I think because both RL-players were pretty bad neither got a chance to develop strategies that would be useful in a game against a human player. If I set the search depth low on the AB-Player I don't think it would be impossible to use it in training the RL-player, even if it would take much longer.

4 Code and methods

All programming was done with Visual Studio Code in the Rust programming language (version 1.67.0). Macroquad was used for GUI graphics (version 0.3). Figure 1 was created using R studio (version 4.1.0).

Link to code: https://github.com/Tucker-Coughlin/Artificial_Intelligence_Final.git

References:

- Russell, S., & Norvig, P. (2022). Artificial intelligence : A modern approach (4th ed., Global ed ed.) Pearson.
- Sutton, R. S., & Barto, A. (2020). Reinforcement learning (Second edition ed.). Cambridge, Massachusetts; London, England: The MIT Press.